

Engenharia de prompt

&

Gestão de contexto

Anatomia de um prompt: system vs user

- O **system prompt** carrega o que não muda a cada chamada: quem o agente é, como ele deve se comportar, em que formato tem que responder.
- O **user prompt** é a parte variável: o código, a pergunta, o dado que precisa ser processado agora.
- Separar os dois é o que permite reaproveitar o mesmo agente em mil situações sem reescrever a lógica toda vez.

```
const systemPrompt = `Você é um revisor de código sênior.  
Responda sempre em JSON, sem texto fora do objeto.`;  
  
const userPrompt = `Revise este trecho:\n${code}`;
```

Prompt ruim vs. prompt bom

Testa esses dois com o mesmo modelo e compara o resultado:

Prompt ruim	Prompt bom
"Revisa esse código"	Papel definido no system prompt + código no user prompt
Não diz o que "revisar" significa aqui	Critérios explícitos: performance, segurança, legibilidade
Resposta em texto solto	Resposta em JSON estruturado, fácil de plugar num pipeline

Prompt vago = saída inconsistente = automação quebrada.

Few-shot prompting

Antes de pedir pro modelo fazer algo novo, mostre exemplos de como você espera que ele faça.

```
const prompt = `
```

```
Exemplo 1:
```

```
Entrada: "função soma dois números"
```

```
Saída: { "nome": "sum", "params": ["a", "b"] }
```

```
Exemplo 2:
```

```
Entrada: "função que valida email"
```

```
Saída: { "nome": "validateEmail", "params": ["email"] }
```

```
Agora sua vez:
```

```
Entrada: "${userInput}"
```

```
Saída: `;
```

Chain-of-thought prompting

Pra tarefas que exigem raciocínio (não só geração de texto), peça pro modelo pensar em voz alta antes de responder.

- "Explique seu raciocínio passo a passo antes de dar a resposta final."
- Funciona bem pra: decisão de arquitetura, debugging, análise de trade-off.
- Força o modelo a não pular direto pra conclusão.
- Origem: WEI et al., 2022, mostraram ganho grande em problema de múltiplos passos só pedindo o raciocínio explícito.

JSON mode: saída estruturada

Se a saída do agente vai alimentar outro pedaço de código (o caso normal na AEP de vocês), texto livre não serve. Vocês precisam de JSON garantido.

```
const response = await model.generateContent({
  contents: [{ role: "user", parts: [{ text: prompt }] }],
  generationConfig: { responseType: "application/json" },
});
```

A própria API garante o formato. Você não depende de o modelo "se comportar bem".

Janela de contexto: o que é, por que estoura

- Todo modelo tem um limite de **tokens** que ele consegue processar de uma vez, prompt + resposta juntos.
- Mandar o repositório inteiro numa pergunta simples estoura esse limite rápido, e ainda custa caro (você paga por token de entrada).
- Contexto grande demais também piora a qualidade da resposta: o modelo "se perde" entre informação relevante e ruído.
- Isso já foi medido: a Chroma testou 18 modelos de ponta em 2025 e todos perderam acerto conforme a entrada crescia, mesmo em tarefa simples de recuperar um trecho.

Usando @ para referenciar arquivos

No Copilot, Cursor e ferramentas parecidas, usar @ seguido do nome do arquivo ou pasta manda só o conteúdo relevante pro modelo, em vez de colar tudo manualmente no chat.

- Peça uma alteração específica referenciando o arquivo exato, não "olha meu projeto inteiro".
- Prefira apontar pra função/trecho isolado quando possível.
- Mesmo raciocínio de um bom code review: contexto cirúrgico gera resposta melhor, e ainda economiza token.

Economia de tokens

- Referência cirúrgica (`@arquivo` , escopo de método) em vez de colar o repositório inteiro.
- Resuma histórico de conversa longa antes de continuar, senão o contexto cresce sem controle.
- Cacheie resposta de prompt repetido. Mesma pergunta duas vezes, cobrança duas vezes.
- Token de entrada e token de saída têm preço diferente, e o preço muda por modelo. Conferir a tabela do provedor antes de estimar custo faz parte do trabalho.

Medindo o custo de uma chamada

- $\text{Custo} = (\text{tokens de entrada} \times \text{preço de entrada}) + (\text{tokens de saída} \times \text{preço de saída})$.
- Não precisa contar token na mão: a própria resposta da API traz a contagem.

```
const res = await model.generateContent(prompt)
const uso = res.response.usageMetadata
console.log(uso.promptTokenCount, uso.candidatesTokenCount)
```

- Logar esses dois números em toda chamada já dá a base de dados de custo e latência que a Entrega 2 pede em média e desvio padrão.

Para estudar mais: prompt e contexto

- Doc oficial, que fica atualizada: platform.claude.com/docs e ai.google.dev.
- WEI et al., 2022, *Chain-of-Thought Prompting*, arxiv.org/abs/2201.11903: paper que originou a técnica.
- LIU et al., 2023, *Lost in the Middle*, arxiv.org/abs/2307.03172: mede a perda de informação no meio do contexto.
- Chroma, 2025, *Context Rot*, research.trychroma.com/context-rot: mesma medição refeita em modelos mais atuais.
- Anthropic, *Effective context engineering for AI agents*, anthropic.com/engineering.